

ARGOS

Adaptive Response Guard with Orchestrated Surveillance
Sprint Semana 1 · Manual del integrante

Sebastian Montenegro

P2 · Ingeniero ML

Tópicos Avanzados de Ciberseguridad · Universidad de Lima · 2026-1
Entrega final: 13 de junio de 2026
Generado: 2026-05-24

Parte I — Introducción al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Field	Value
Type	Manual de introducción común para los 4 integrantes
Status	Active
Última actualización	2026-05-24
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es para el curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30% del peso), demo en vivo (~40%) y presentación (~20%) más los seguimientos en semanas 5/7/9 (~10%).

El activo defendido es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como `criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El alcance multi-vector (post ADR-0008) cubre tres categorías de ataques distintas:

- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL

es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.



Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: `file_write_rate`, `avg_entropy` (Shannon), `extension_modification_ratio`, `crypto_api_calls`, `new_outbound_connections`, `cpu_burst_score`, `io_burst_score`.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: `connections/sec`, `packet rate`, `source IP entropy`, `packet size variance`.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: `rows_returned`, `query_duration_ms`, `hour_of_day`, `user_id`, `query_template_hash`.

Ensemble ransomware: `ensemble_score = 0.6 × isolation_forest_score + 0.4 × one_class_svm_score`. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si halucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
☐ T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
☐ T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
☐ T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
☐ T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md §Q5`.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
☐	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
☐	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
☐	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
☐	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --speed full
```


Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.
- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: `file_write_rate=347` archivos/min, `avg_entropy=7.8` (alta), `extension_modification_ratio=0.95`, `crypto_api_calls=42`, `cpu_burst_score=0.9`.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).

4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el `AlertContext` con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve `TriageResponse`: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.
- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block`.
2. **Process kill**: identifica el PID del simulator desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>`.
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED`.

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 `chat_ids` configurados: mensaje con `incident_id`, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh`. Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, `incident_id`, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED`, decisión `EXECUTE_ISOLATION`, política aplicada `auto-execute`, justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: `alert created` → `tier classified` → `playbook executed` → `notifications sent`.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/ archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+	Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US- based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+		

Categoría	Componente	Función	Owner principal
		Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- **Branch principal:** `main`, protegida en GitHub.
- **Feature branches:** `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- **Commits convencionales:** `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- **PRs requieren:** CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- **Push diario obligatorio** antes de las 22:00 durante el sprint.

Estructura del repo (post ADR-0008)


```

argos/
├── README.md           # Entry point del proyecto
├── LICENSE             # MIT
├── pyproject.toml      # Metadata + extras por módulo
├── .env.example        # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/    # □ Pydantic v2 cross-team (entregado · v1.1.0 · 69 tests)
│   ├── _mitre_data.py  # MITRE_WHITELIST curado
│   ├── alert.py        # WazuhAlert, NormalizedAlert
│   ├── ml_score.py     # MLScore (P2)
│   ├── triage.py       # AlertContext, TriageResponse (P1)
│   ├── incident.py     # Incident state machine (P1)
│   ├── approval.py     # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py        # Tier, Severity, NotificationChannelType, ...
│   └── tests/          # 69 validation tests
├── llm_triage/         # P1 – Capa 4 LLM Triage
│   ├── api/main.py     # FastAPI /triage endpoint
│   ├── llm_client/     # OpenAI primary + Llama local fallback
│   ├── rag/            # BM25 + BGE-large + RRF
│   └── prompts/        # Jinja2 templates
├── soar/              # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/ # tier_classifier, state_machine, orchestrator
│   ├── approval/       # api, jwt_signer, consolidation
│   ├── notification/   # telegram, discord, twilio_voice, email
│   └── playbooks/      # host_isolation, process_kill, snapshot, throttle
├── ml/                # P2 – Capa 2 ML
│   ├── features/       # Feature extractors por dominio
│   ├── models/         # ransomware_ensemble, network_traffic, query_pattern
│   └── consumer/       # Redis stream consumer
├── detection/         # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/ # T1486, T1490, T1083, T1562
│   │   ├── network/    # T1498, T1499 rate-based
│   │   ├── database/   # query patterns
│   │   └── webapp/     # T1190 SQLi signatures
│   └── wazuh-rules/    # Convertidas con sigma-cli
├── deception/         # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/      # Wazuh FIM rules
├── ui/                # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/ # JSON dashboards exportados
├── attack-simulation/ # Simuladores multi-vector
│   ├── ransomware_simulator/ # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/     # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/      # P4 (sqlmap)
│   └── pg_simulator/        # P4 (SELECT masivo para UC-07)
├── lab/               # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/      # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/        # P2 + P4 – Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md   # 90-second overview
│   ├── CONTEXT.md        # Onboarding completo
│   ├── PROJECT_STATUS.md  # Honest snapshot
│   ├── EVALUATION_CRITERIA.md # Rúbrica del curso
│   ├── data-handling.md   # T-030 sanitization policy
│   └── README.md          # Mapa de docs

```

```

├── architecture/      # SAD, threat model, flow diagrams
├── decisions/         # 8 ADRs + OPEN_QUESTIONS
├── use-cases/         # 8 UCs detallados
└── team/              # Sprint manuals (este documento + 4 individuales)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual. Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).
- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde `NNN` es contador diario zero-padded a 3 dígitos. Ejemplo: `INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar el sprint. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

```

cd ~/projects                # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos

```

Paso 2 — Crear virtualenv Python

```

python3 -m venv .env
source .env/bin/activate      # Linux/macOS
# .env\Scripts\activate       # Windows PowerShell
pip install -U pip setuptools wheel

```

Paso 3 — Instalar dependencias core + tu rol

```

# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"    # P1 (siempre + más abajo)
pip install -e ".[ml]"           # P2
pip install -e ".[soar,llm]"     # P1 (servicios)
pip install -e ".[ui]"           # P4

```

Paso 4 — Verificar contracts

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/sprint-w1-init
```

11. Comunicación del equipo durante el sprint

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup` , llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato (per `docs/team/standup-template.md`):**
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc esta semana.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.
2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning en semana 2.

3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega en esta semana 1

Para que no te sorprendas cuando llegue Día 7 sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas). Semana 2.
- **UC-03 split-brain con 4 aprobadores reales** no scripted. Semana 2-3.
- **UC-05 stealth attack** end-to-end pulido. Semana 2.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos). Semana 2-3.
- **Video demo final editado.** Semana 3.
- **Informe técnico final.** Semana 3.
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa al cierre del sprint Día 7: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, con dos rehearsals al domingo.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual de sprint individual	docs/team/sprint-week-1-p<X>-<nombre>.md
Plan operacional general del sprint	docs/team/sprint-week-1-overview.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual individual de Sebastian Montenegro

Sprint Semana 1 — Manual de P2 (Sebastian Montenegro)

Field	Value
Owner	Sebastian Montenegro
Rol	P2 · Ingeniero ML
Goal de la semana	Capa 2 ML completa (ensemble ransomware Isolation Forest + One-Class SVM) + modelos especializados para UC-06 network y UC-07 query patterns + pipeline Redis consumer + métricas A/B/C + captura forense
Effort estimado	6 horas reales/día × 7 días = 42 horas
Pre-requisitos	Leer docs/team/sprint-week-1-common-intro.md y docs/decisions/0008-multi-vector-scope-expansion.md

Antes de empezar — chequeo de prerequisites

Hardware

- Laptop con **mínimo 16 GB RAM** (entrenar modelos ML + Jupyter + IDE come 12 GB fácil).
- 30 GB disco libre para datasets y modelos.
- macOS / Linux / Windows con WSL2.

Software base

```
python3 --version    # 3.11+
git --version
# Jupyter Lab opcional pero útil para EDA
```

Cuentas externas

Para P2 no necesitas cuentas externas adicionales. Tu trabajo es 100% local + acceso al repo. P1 te compartirá credenciales si necesitas tocar Redis/OpenSearch del lab de demo, pero al inicio del sprint puedes usar fakeredis para todo.

Día 1 (Lunes) — Setup ambiente ML + investigación de baseline

Goal del día: entorno Python listo, scikit-learn + scipy + pandas funcionando, primer Isolation Forest entrenado con datos sintéticos, plan de baseline real definido.

Tiempo: 5-6 horas.

Paso 1.1 — Clonar repo y crear venv (15 min)

```
cd ~/projects
git clone https://github.com/EnzoOrdenez/argos.git
cd argos

python3 -m venv .venv
source .venv/bin/activate          # Linux/macOS
# .venv\Scripts\activate           # Windows

pip install -U pip setuptools wheel
pip install -e ".[ml,dev]"
```

Verificación:

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
python -c "import sklearn, scipy, pandas, numpy, joblib; print('ML stack OK')"
```

Paso 1.2 — Crear estructura del módulo ml/

```
mkdir -p ml/features ml/models ml/consumer ml/notebooks ml/tests
touch ml/__init__.py ml/features/__init__.py ml/models/__init__.py ml/consumer/__init__.py ml/tests/__init__.py
```

Paso 1.3 — Investigar features para ransomware (1.5 h)

Lee:

- SAD §5.2 (features ventana 60s)
- ADR-0008 §"Capa 2 ML — modelos especializados"
- Surveys de ML anomaly detection para ransomware (papers IEEE 2023-2024)

Las 7 features ya definidas en `argos_contracts/ml_score.py` → `MLFeatures` :

```
file_write_rate: float          # archivos escritos por minuto
avg_entropy: float              # Shannon entropy promedio del contenido escrito
extension_modification_ratio: float # ratio de archivos con extensión cambiada
crypto_api_calls: int           # CryptEncrypt, BCryptEncrypt (Windows); openssl/libsodium hooks (Linux)
new_outbound_connections: int    # conexiones C2 sospechosas
cpu_burst_score: float           # picos de CPU sostenidos
io_burst_score: float            # picos de I/O sostenidos
```

Paso 1.4 — Feature extractor con datos sintéticos (2 h)

Crea `ml/features/extractor.py` :


```

"""Feature extraction per process per 60s window for ransomware detection.

Reference: SAD §5.2.
"""
import math
from collections import Counter
from dataclasses import dataclass
from pathlib import Path

from argos_contracts import MLFeatures

@dataclass
class ProcessTelemetry:
    """Telemetría capturada para un proceso en una ventana de 60s."""
    pid: int
    process_name: str
    files_written: list[Path]
    file_contents_sample: dict[Path, bytes] # primeros 4KB de cada archivo
    extensions_before: list[str]
    extensions_after: list[str]
    crypto_api_calls_count: int
    outbound_connections_new: int
    cpu_samples: list[float] # uso CPU% cada 5s
    io_read_bytes: int
    io_write_bytes: int
    window_duration_s: float = 60.0

def shannon_entropy(data: bytes) -> float:
    """Entropía Shannon de un blob binario, normalizada a [0, 8]."""
    if not data:
        return 0.0
    counter = Counter(data)
    length = len(data)
    return -sum(
        (count / length) * math.log2(count / length)
        for count in counter.values()
        if count > 0
    )

def extract_features(t: ProcessTelemetry) -> MLFeatures:
    """Convierte ProcessTelemetry en MLFeatures (Pydantic contract)."""
    # File write rate (archivos/min)
    file_write_rate = len(t.files_written) / (t.window_duration_s / 60.0)

    # Average entropy
    entropies = [shannon_entropy(content) for content in t.file_contents_sample.values()]
    avg_entropy = sum(entropies) / len(entropies) if entropies else 0.0

    # Extension modification ratio
    if not t.extensions_before:
        extension_modification_ratio = 0.0
    else:
        modified = sum(
            1 for before, after in zip(t.extensions_before, t.extensions_after)
            if before != after
        )
        extension_modification_ratio = modified / len(t.extensions_before)

    # CPU burst score (max sostenido)
    cpu_burst_score = max(t.cpu_samples) / 100.0 if t.cpu_samples else 0.0

    # I/O burst score (proxy: write bytes / duration)
    io_burst_score = min(1.0, t.io_write_bytes / (10_000_000 * t.window_duration_s)) # >10MB/s = 1.0

    return MLFeatures(
        file_write_rate=file_write_rate,
        avg_entropy=avg_entropy,
        extension_modification_ratio=extension_modification_ratio,
        crypto_api_calls=t.crypto_api_calls_count,
        new_outbound_connections=t.outbound_connections_new,
    )

```

```

        cpu_burst_score=cpu_burst_score,
        io_burst_score=io_burst_score,
    )

```

Test rápido:

```

python -c "
from ml.features.extractor import ProcessTelemetry, extract_features
t = ProcessTelemetry(
    pid=1234, process_name='ransom.exe',
    files_written=[],
    file_contents_sample={},
    extensions_before=['.txt', '.docx'],
    extensions_after=['.txt.locked', '.docx.locked'],
    crypto_api_calls_count=42,
    outbound_connections_new=1,
    cpu_samples=[90.0, 95.0, 92.0],
    io_read_bytes=0,
    io_write_bytes=50_000_000,
)
features = extract_features(t)
print(features.model_dump_json(indent=2))
"

```

Paso 1.5 — Primera Isolation Forest con datos sintéticos (1.5 h)

Crea `ml/models/train_baseline.py`:

```

"""Entrenamiento inicial de Isolation Forest sobre datos sintéticos.

Esto es Day 1 – los datos son sintéticos para validar el pipeline.
Día 3 entrenamos sobre baseline real del lab.
"""
from pathlib import Path

import joblib
import numpy as np
from sklearn.ensemble import IsolationForest

def generate_synthetic_baseline(n_samples: int = 1000, seed: int = 42) -> np.ndarray:
    """Genera 1000 muestras de actividad benigna sintética.

    Cada muestra es un vector de 7 features (orden de MLFeatures).
    """
    rng = np.random.default_rng(seed)
    return np.column_stack([
        rng.normal(5, 2, n_samples).clip(0, None),      # file_write_rate baseline ~5/min
        rng.normal(4.0, 0.5, n_samples).clip(0, 8),    # avg_entropy benigna ~4 (texto normal)
        rng.beta(1, 10, n_samples),                    # extension_modification_ratio benigna ~0.1
        rng.poisson(2, n_samples),                      # crypto_api_calls benignas ~2
        rng.poisson(1, n_samples),                      # new_outbound_connections ~1
        rng.beta(2, 5, n_samples),                      # cpu_burst_score benigno ~0.3
        rng.beta(2, 5, n_samples),                      # io_burst_score benigno ~0.3
    ])

def train_isolation_forest(X: np.ndarray, n_estimators: int = 100) -> IsolationForest:
    """Isolation Forest con contamination 0.1 (10% outliers esperados)."""
    model = IsolationForest(
        n_estimators=n_estimators,
        contamination=0.1,
        random_state=42,
        n_jobs=-1,
    )
    model.fit(X)
    return model

if __name__ == "__main__":
    X = generate_synthetic_baseline(n_samples=1000)
    model = train_isolation_forest(X)
    output = Path("ml/models/iforest_synthetic_v0.joblib")
    output.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, output)
    print(f"Saved {output} (n_estimators=100, contamination=0.1)")

    # Sanity check: predict sobre un caso ransomware
    ransom = np.array([[120, 7.8, 0.95, 42, 3, 0.9, 0.95]]) # ransomware features
    score = model.score_samples(ransom)[0]
    print(f"Ransom case score: {score:.3f} (negativo = más anómalo)")

```

```

python ml/models/train_baseline.py
# Esperado: archivo creado + score muy negativo (ej. -0.4)

```

Paso 1.6 — Definir plan de baseline real (45 min)

Crea `ml/notebooks/00_baseline_plan.md`:

```
# Plan de recolección de baseline benigno

**Objetivo:** capturar ~2 semanas de actividad benigna real del lab para entrenar Isolation Forest + One-Class SVM sol

**Actividades benignas a simular:**
1. Editar archivos de texto (notepad, vim) → file_write_rate bajo, avg_entropy bajo
2. Compresión con 7zip / tar → write rate medio, entropy ALTO (este es un FP típico!)
3. Backup con duplicati o restic → write rate medio, muchos archivos
4. Update de sistema (apt upgrade) → muchos files modified
5. Browsing web (descargar archivos) → new outbound connections altas
6. Compilación de software → CPU/IO burst alto

**Captura:** P4 levantará Wazuh y yo conectaré mi consumer al stream Redis durante la semana de baseline.

**Deliverable:** dataset CSV con ~10K muestras etiquetadas como BENIGN.
```

Paso 1.7 — Commit + PR (10 min)

```
git checkout -b feature/p2/ml-features-baseline
git add ml/
git commit -m "feat(p2): feature extractor + IF synthetic baseline v0"
git push origin feature/p2/ml-features-baseline
```

Verificación EOD Día 1

- ☐ `pytest argos_contracts/tests/` pasa 69 tests
- ☐ `python ml/models/train_baseline.py` crea archivo .joblib
- ☐ Score de caso ransomware es claramente negativo
- ☐ PR abierto

Bloqueos comunes Día 1

Problema	Causa	Fix
<code>ImportError: No module named sklearn</code>	venv no activado o [ml] no instalado	<code>source .venv/bin/activate && pip install -e ".[ml,dev]"</code>
<code>IsolationForest.score_samples</code> returns NaN	datos con NaN	Limpieza: <code>X = X[~np.isnan(X).any(axis=1)]</code>

Día 2 (Martes) — One-Class SVM + ensemble + Redis consumer

Goal: OC-SVM entrenado, ensemble combinando ambos modelos, Redis consumer que lee alertas Wazuh y publica MLScore.

Tiempo: 6 horas.

Paso 2.1 — One-Class SVM training (1.5 h)

Añade a `ml/models/train_baseline.py`:

```
from sklearn.svm import OneClassSVM

def train_one_class_svm(X: np.ndarray, nu: float = 0.1) -> OneClassSVM:
    """One-Class SVM con kernel RBF.

    nu = fraction esperada de outliers (0.1 = 10%).
    """
    model = OneClassSVM(kernel="rbf", nu=nu, gamma="scale")
    model.fit(X)
    return model
```

Paso 2.2 — Ensemble (1 h)

Crea `ml/models/ensemble.py`:

```
"""Ensemble Isolation Forest (0.6) + One-Class SVM (0.4) per SAD §5.2."""
import joblib
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM

class RansomwareEnsemble:
    """Combina IF + OC-SVM en un score [0,1]."""

    IF_WEIGHT = 0.6
    SVM_WEIGHT = 0.4

    def __init__(self, iforest: IsolationForest, ocsvm: OneClassSVM):
        self.iforest = iforest
        self.ocsvm = ocsvm

    def score(self, X: np.ndarray) -> tuple[float, float, float]:
        """Returns (iforest_score, svm_score, ensemble_score) en [0,1]."""
        # Isolation Forest: score_samples returns higher = more normal. Convertimos a anomaly score.
        if_raw = self.iforest.score_samples(X)
        if_anomaly = 1.0 - self._normalize(if_raw)

        # OC-SVM: decision_function returns higher = more normal.
        svm_raw = self.ocsvm.decision_function(X)
        svm_anomaly = 1.0 - self._normalize(svm_raw)

        ensemble = self.IF_WEIGHT * if_anomaly + self.SVM_WEIGHT * svm_anomaly
        return float(if_anomaly[0]), float(svm_anomaly[0]), float(ensemble[0])

    @staticmethod
    def _normalize(scores: np.ndarray) -> np.ndarray:
        """Min-max scaling – para Day 2 con rango fijo asumido."""
        # Rangos típicos observados durante calibración
        return np.clip((scores + 0.5) / 1.0, 0.0, 1.0)

    def save(self, path: str) -> None:
        joblib.dump({"iforest": self.iforest, "ocsvm": self.ocsvm}, path)

    @classmethod
    def load(cls, path: str) -> "RansomwareEnsemble":
        data = joblib.load(path)
        return cls(iforest=data["iforest"], ocsvm=data["ocsvm"])
```

Paso 2.3 — Redis consumer (2 h)

Crea `ml/consumer/consumer.py`:

```
"""Consume alertas Wazuh desde Redis stream, extrae features, publica MLScore."""
import asyncio
import json
from datetime import datetime, timezone

import redis.asyncio as redis

from argos_contracts import MLScore, NormalizedAlert
from ml.features.extractor import extract_features
from ml.models.ensemble import RansomwareEnsemble

class MLConsumer:
    def __init__(self, redis_url: str = "redis://localhost:6379/0", ensemble_path: str = "ml/models/ransomware_ensemble"):
        self.redis = redis.from_url(redis_url, decode_responses=True)
        self.ensemble = RansomwareEnsemble.load(ensemble_path)

    async def run(self, stream: str = "wazuh:alerts", group: str = "ml-consumers", consumer: str = "p2-1"):
        # Create consumer group si no existe
        try:
            await self.redis.xgroup_create(stream, group, id="$", mkstream=True)
        except redis.ResponseError as e:
            if "BUSYGROUP" not in str(e):
                raise

        while True:
            messages = await self.redis.xreadgroup(
                group, consumer, {stream: ">"}, count=10, block=5000,
            )
            for _stream, entries in messages:
                for entry_id, fields in entries:
                    await self._process(fields, entry_id, stream, group)

    async def _process(self, fields: dict, entry_id: str, stream: str, group: str):
        try:
            alert = NormalizedAlert.model_validate_json(fields["data"])
            # ... extract features from alert telemetry (Day 3 wires Wazuh process_info correctly)
            # Por ahora generamos features de prueba:
            import numpy as np
            X = np.array([[10.0, 5.0, 0.2, 5, 1, 0.4, 0.4]])
            if_score, svm_score, ensemble = self.ensemble.score(X)

            score = MLScore(
                score_id=f"score-{alert.alert_id}",
                timestamp=datetime.now(timezone.utc),
                host_id=alert.host_id,
                isolation_forest_score=if_score,
                one_class_svm_score=svm_score,
                ensemble_score=ensemble,
                features=..., # MLFeatures real
                model_version="ransomware-ensemble-v0",
            )
            await self.redis.xadd("ml:scores", {"data": score.model_dump_json()})
            await self.redis.xack(stream, group, entry_id)
        except Exception as e:
            print(f"Error processing {entry_id}: {e}")

if __name__ == "__main__":
    asyncio.run(MLConsumer().run())
```

Paso 2.4 — Tests con fakeredis (1 h)

`ml/tests/test_consumer.py`:

```
import pytest
from fakeredis import aioredis as fakeredis_async
# ... test que el consumer lee alerts del stream y publica MLScore
```

Paso 2.5 — Commit (10 min)

```
git add ml/
git commit -m "feat(p2): One-Class SVM + ensemble + Redis consumer (ransomware)"
git push
```

Verificación EOD Día 2

- ☐ Ensemble retorna scores válidos en [0,1]
- ☐ Consumer con fakeredis no crashea
- ☐ PR actualizado

Día 3 (Miércoles) — Baseline real del lab + retraining

Goal: P4 tiene Wazuh manager arriba. P2 conecta consumer al lab real, recolecta baseline (~4 horas de actividad benigna simulada), reentrena ensemble.

Tiempo: 6 horas.

Paso 3.1 — Coordinar con P4 (15 min)

Mensaje en `#argos-help`: "Hey P4, necesito acceso al Redis del lab para conectar mi ML consumer. ¿Me das el endpoint del stream?"

P4 te responde con el `LAB_MANAGER_IP` del Vagrantfile (típicamente `10.0.0.10`) y el puerto Redis (6379).

Paso 3.2 — Conectar consumer al lab (1 h)

Edita `.env` local:

```
REDIS_HOST=10.0.0.10
REDIS_PORT=6379
```

Levanta consumer:

```
python -m ml.consumer.consumer
```

P4 ejecuta actividad benigna en las VMs víctima (browsing, file edits, etc.). Tú observas que llegan alertas al stream y tu consumer las procesa.

Paso 3.3 — Recolectar baseline (2-4 h pasivas)

Mientras corre el consumer, tú trabajas en otra cosa (research papers, ablation prep). El consumer va llenando un CSV con features observadas:

```
# ml/consumer/consumer.py, añadir:
import csv

def __init__(self, ..., baseline_csv: str = "ml/data/baseline.csv"):
    ...
    self.baseline_writer = csv.writer(open(baseline_csv, "a"))
```

Al final de ~4 horas tienes ~500-1000 muestras benignas reales.

Paso 3.4 — Retraining sobre baseline real (1 h)

```
import pandas as pd
df = pd.read_csv("ml/data/baseline.csv", names=["file_write_rate", "avg_entropy", ...])
X = df.values
ensemble = RansomwareEnsemble(
    iforest=train_isolation_forest(X),
    ocsvm=train_one_class_svm(X),
)
ensemble.save("ml/models/ransomware_ensemble.pkl")
```

Paso 3.5 — Commit (10 min)

```
git add ml/models/ransomware_ensemble.pkl ml/data/baseline.csv
git commit -m "feat(p2): baseline real recolectado + ensemble retrained"
git push
```

Verificación EOD Día 3

- ☐ Consumer conecta al Redis del lab sin errores
- ☐ Baseline CSV tiene >500 muestras reales
- ☐ Ensemble retrained, archivo .pkl actualizado

Día 4 (Jueves) — Modelo network anomaly (UC-06)

Goal: Modelo ML especializado para UC-06 DDoS. Features distintas: connections/sec, packet rate, source IP entropy.

Tiempo: 6 horas.

Paso 4.1 — Features de network traffic (1.5 h)

```
# ml/features/network_features.py
@dataclass
class NetworkWindowTelemetry:
    connections_count: int
    packets_count: int
    unique_source_ips: int
    avg_packet_size: float
    syn_count: int
    duration_s: float = 60.0

def extract_network_features(t: NetworkWindowTelemetry):
    return {
        "connections_per_sec": t.connections_count / t.duration_s,
        "packets_per_sec": t.packets_count / t.duration_s,
        "src_ip_entropy": math.log2(t.unique_source_ips) if t.unique_source_ips > 1 else 0.0,
        "avg_packet_size": t.avg_packet_size,
        "syn_ratio": t.syn_count / max(t.packets_count, 1),
    }
```

Paso 4.2 — Entrenar `network_traffic_anomaly.pkl` (2 h)

```
# Synthetic network baseline
benign_network = np.array([
    rng.normal(50, 20, n).clip(0),      # connections/sec normal
    rng.normal(500, 200, n).clip(0),    # packets/sec normal
    rng.beta(2, 8, n) * 10,             # src IP entropy bajo (pocos clientes únicos)
    rng.normal(800, 300, n).clip(0),    # avg packet size normal
    rng.beta(1, 20, n),                 # SYN ratio bajo
])

# DDoS sería: 10000 connections/sec, 100000 packets/sec, src_ip_entropy=15 (muchas IPs), SYN ratio=0.9
network_model = train_isolation_forest(benign_network.T)
joblib.dump(network_model, "ml/models/network_traffic_anomaly.pkl")
```

Paso 4.3 — Wiring con SOAR (coordinar con P1) (1 h)

P1 necesita que tu MLScore tenga un campo identificador del dominio. Usar `model_version` para esto: `"ransomware-ensemble-v1"` vs `"network-anomaly-v1"`.

Paso 4.4 — Tests + commit (1 h)

```
git commit -m "feat(p2): network traffic anomaly model for UC-06"
```

Verificación EOD Día 4

- ☐ Modelo network entrenado
- ☐ Caso DDoS sintético da score >0.9

Día 5 (Viernes) — Modelo query patterns (UC-07)

Goal: Modelo ML para detectar SELECT masivo anómalo. **Este es el modelo más importante** porque alimenta UC-07 (la pieza clave del HITL).

Tiempo: 7 horas (el día más largo).

Paso 5.1 — Features de query patterns (2 h)

```
# ml/features/query_features.py
@dataclass
class QueryWindowTelemetry:
    rows_returned: int
    query_duration_ms: float
    hour_of_day: int
    user_id: str
    query_template_hash: str
    bytes_returned: int

def extract_query_features(t: QueryWindowTelemetry):
    return {
        "log_rows_returned": math.log10(max(t.rows_returned, 1)),
        "log_duration_ms": math.log10(max(t.query_duration_ms, 1)),
        "hour_sin": math.sin(2 * math.pi * t.hour_of_day / 24),
        "hour_cos": math.cos(2 * math.pi * t.hour_of_day / 24),
        "user_hash": hash(t.user_id) % 1000 / 1000,
        "bytes_per_row": t.bytes_returned / max(t.rows_returned, 1),
    }
```

Paso 5.2 — Coordinar con P4 sobre pgAudit (30 min)

P4 instala pgAudit en PostgreSQL VM. Cada query loggeada tiene: timestamp, user, query, rows_returned, duration. P2 recibe estos como stream Redis `pg:queries`.

Paso 5.3 — Entrenar baseline de queries normales (2 h)

Genera 1000 queries sintéticas "normales":

- SELECT pequeño (10-100 filas), duration <1s, horario laboral, usuarios variados
- INSERT/UPDATE pequeños
- Algunos JOIN moderados (1000-5000 filas)

Entrena Isolation Forest sobre esas 1000 muestras.

Paso 5.4 — Test escenario UC-07 (1 h)

```
# UC-07: Sebastian a las 3 AM hace SELECT que devuelve 100K filas
uc07_case = np.array([
    math.log10(100000), # log_rows_returned = 5
    math.log10(8000), # log_duration_ms = 3.9
    math.sin(2 * math.pi * 3 / 24), # hour_sin (3 AM)
    math.cos(2 * math.pi * 3 / 24), # hour_cos
    hash("sebastian") % 1000 / 1000, # user_hash
    50, # bytes_per_row
])
score = query_model.score_samples(uc07_case)
print(f"UC-07 score: {score}") # debe ser anómalo pero no extremo (~ -0.3)
```

Paso 5.5 — Commit (10 min)

```
git commit -m "feat(p2): query pattern anomaly model for UC-07"
```

Verificación EOD Día 5

- ☐ Modelo query trained
- ☐ Caso UC-07 (SELECT masivo) sale con anomaly score ~0.65 (medio, T2 territory)
- ☐ Caso normal (SELECT pequeño en horario laboral) sale con score bajo (~0.2)

Día 6 (Sábado) — Captura forense + métricas

Goal: Pipeline de captura forense (process tree, hashes, command-line) + computación de métricas A/B/C.

Tiempo: 6 horas.

Paso 6.1 — Forensic capture pipeline (2 h)

```
# evaluation/forensics/capture.py
import hashlib
import psutil

def capture_process_tree(pid: int) -> dict:
    """Snapshot del process tree alrededor del PID ofensor."""
    proc = psutil.Process(pid)
    return {
        "pid": pid,
        "name": proc.name(),
        "cmdline": proc.cmdline(),
        "parent_pid": proc.ppid(),
        "children": [{"pid": c.pid, "name": c.name()} for c in proc.children(recursive=True)],
        "open_files": [f.path for f in proc.open_files()],
        "network_connections": [
            {"local": str(c.laddr), "remote": str(c.raddr), "status": c.status}
            for c in proc.net_connections()
        ],
    }

def hash_files(paths: list[str]) -> dict[str, str]:
    """SHA-256 de cada archivo."""
    return {p: hashlib.sha256(open(p, "rb").read()).hexdigest() for p in paths}
```

Paso 6.2 — Métricas A/B/C (3 h)

evaluation/metrics/compute.py :

- **A. Demo headline:** TTD, files affected, FP rate
- **B. Forensic timeline:** complete event chain
- **C. System evaluation:** P/R/F1 per layer, MITRE coverage, ablation

```
def compute_pr_f1(predictions, ground_truth):
    from sklearn.metrics import precision_recall_fscore_support
    return precision_recall_fscore_support(ground_truth, predictions, average="binary")
```

Paso 6.3 — Commit (10 min)

Verificación EOD Día 6

- ☐ Forensic capture funciona contra un PID real
 - ☐ Métricas P/R/F1 calculan sobre dataset sintético
-

Día 7 (Domingo) — Rehearsals + calibración inicial

Mañana (9-13h): rehearsals con P1+P3+P4. Tu rol es asegurar que ML scores aparecen correctamente en la Streamlit Console durante UC-01, UC-03, UC-06, UC-07.

Tarde (14-18h): bug bash. Bugs típicos del ML:

- Threshold demasiado bajo → muchos false positives
- Modelo no entrenado con suficientes datos → scores erráticos
- Feature normalization rota cuando cambian rangos

Noche (19-21h): documentar status en `evaluation/notes/sprint-w1-status.md`.

Entregable EOD Día 7

- ☐ 4 modelos entrenados (ransomware + network + query + opcional NSL-KDD para EV)
 - ☐ Métricas iniciales calculadas (P/R/F1 sobre synthetic + baseline real)
 - ☐ PR final mergeado
-

Apéndice A — Comandos diarios

```
# Activar env
cd ~/projects/argos && source .venv/bin/activate

# Levantar consumer
python -m ml.consumer.consumer

# Reentrenar ensemble
python ml/models/train_baseline.py

# Tests rápidos
pytest argos_contracts/tests/ ml/tests/ -x

# Limpiar fakeredis state
redis-cli -h 10.0.0.10 FLUSHDB

# Jupyter para EDA
jupyter lab ml/notebooks/
```

Apéndice B — Troubleshooting

Síntoma	Causa	Fix
<code>ImportError: sklearn</code>	venv no activado	<code>source .venv/bin/activate && pip install -e ".[ml]"</code>
Modelo predice todo benigno	contamination demasiado bajo	Subir a 0.15-0.2
Score muy errático	features no normalizadas	Aplicar StandardScaler antes del train
Redis connection refused	Lab P4 no levantado	Coordinar con P4
<code>MLScore validation error</code>	features incompletas	Verificar todos los 7 campos en MLFeatures

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial manual P2 — Capa 2 ML completa + modelos especializados UC-06/07 + forensics + métricas.	P1